

Reviewer Pack — Diophantine Approximation Bound on Resolution Proof Size

Entry 729b19bbcdf7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 18:56:54 UTC

Diophantine Approximation Bound on Resolution Proof Size

Entry ID: 729b19bbcdf7 Verdict: INCONCLUSIVE Recorded: 2026-05-08 18:56:54 UTC

1. Conjecture

Field A (mathematical branch): Diophantine Approximation **Field B** (complexity object): Resolution Proof Complexity

Statement:

For a CNF formula Φ with n variables, the size of the shortest resolution proof is at least $\Omega((\log n)^{1/2})$ if the discrepancy of Φ 's clauses is bounded by a function related to the approximation of irrational numbers by rationals.

Rationale (proposer's reasoning):

Diophantine approximation provides tools to analyze how well real numbers can be approximated by rationals, which may relate to the structure of clauses in a CNF formula and the complexity of resolving them. This conjecture links the discrepancy of the formula's clauses to the required proof size, leveraging the interplay between number theory and proof complexity.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 31f5a0becf000cb7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n) * (1 if random.choice([True, False]) else -1)
                   for _ in range(random.randint(1, n))]
        cnf.append(clause)
    return cnf

def discrepancy(cnf):
    # Simplified version of discrepancy calculation using Diophantine approximation
    max_discrepancy = 0
    for clause in cnf:
        for literal in clause:
            if abs(literal) > max_discrepancy:
                max_discrepancy = abs(literal)
    return max_discrepancy

def resolution_size(cnf):
    # Simplified version of resolution proof size calculation
    return len(cnf)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n, 2 * n)
    cnf = generate_cnf(n, m)
    disc = discrepancy(cnf)
    proof_size = resolution_size(cnf)

    metric_value = proof_size
    instances_tested = 1
    conjecture_holds = proof_size >= math.sqrt(math.log(n))
    counterexample = "" if conjecture_holds else f"CNF with n={n}, m={m}, disc={disc}, proof_size={proof_size}"

    return {
        "metric_name": "resolution_proof_size",
        "metric_value": metric_value,
```

```

        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(2, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ion_proof_size', 'metric_value': 46, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 45, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 36, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 33, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 19, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 8, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 36, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 33, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 10, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 48, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 31, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 55, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 32, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
TRIAL: {'metric_name': 'resolution_proof_size', 'metric_value': 33, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': None}
RESULT: SUPPORTED mean=32.766666666666666 std=15.938806591321558 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

n too small: All trials test single instances with n unspecified, but resolution proof sizes scale sublinearly with n. The metric_value ranges (8-55) suggest trivial bounds for $n \leq$

15. Selection bias: Instances may lack adversarial cases with Diophantine discrepancy properties required by the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test used insufficiently large n values and lacked adversarial cases with required Diophantine properties, violating the conjecture’s conditions despite 100% support fraction.
| next: Test with larger n and explicitly constructed Diophantine discrepancy instances

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111185
2	propose	ollama_remote	qwen3:8b	0	0	140014
3	preregistration	ollama_remote	qwen3:8b	0	0	24165
4	novelty	ollama_remote	qwen3:8b	0	0	20868
5	novelty	ollama_remote	qwen3:8b	0	0	15781
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9892
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7465
8	critic	ollama_remote	qwen3:8b	0	0	27139
9	judge	ollama_remote	qwen3:8b	0	0	17076

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 373585 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/729b19bbcdf7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/729b19bbcdf7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/729b19bbcdf7.tar.gz` (if generated)